Batch: A - 3	**Roll No.: 16010124073**
Experiment / assignment / tutorial No. 1	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title: Advanced JavaScript Concepts

AIM: Implementation of Java script and advanced JavaScript Concepts.

Objective:

- Understand and implement key advanced JavaScript concepts.
- Learn how the JavaScript engine executes code (execution context, call stack).
- Explore modern ES6+ features like arrow functions, destructuring, and template literals.
- Apply asynchronous programming concepts using Promises and async/await.
- Use closures, callbacks, and higher-order functions effectively.
- Understand and utilize prototypal inheritance and object-oriented JavaScript.

Problem Definition:

JavaScript is a core language for web development, enabling dynamic and interactive user experiences. While many developers are familiar with its basics, mastering advanced JavaScript concepts is crucial for building scalable, performant, and maintainable applications.

This assignment aims to deepen understanding of advanced JavaScript features such as closures, promises, async/await, prototypes, higher-order functions, ES6+ features, and module patterns. Through hands-on implementation, debugging, and real-world scenarios, students will develop the confidence to write clean, efficient, and modern JavaScript code..

Resources used:

https://javascript.info/

Expected OUTCOME of Experiment:

CO 4: Illustrate the concepts of various front-end, back-end web application development technologies & frameworks using different web development tools.

Books/ Journals/ Websites referred:

1. Shelly Powers Learning Node O' Reilly 2 nd Edition, 2016.

Pre Lab/ Prior Concepts:

- **Basics of JavaScript syntax**
- **Variables (let, const)**
- **Data types**
- **Functions**
- **Arrays and Objects**
- **Loops and Conditional Statements**
- **ES6 Features**
- **Node.js execution using VS Code**

Methodology:

- Read and understood each question from the worksheet.
- Analyzed the required JavaScript concept.
- Implemented the solution using modern JavaScript (ES6+) in VS Code.
- Executed each program using Node.js.
- Verified the output with the expected result.
- Captured screenshots of the execution.
- Compiled all solutions into the lab record.

Implementation Details:

Advanced JavaScript Practice Worksheet- Solutions

Array Methods

Question 1

1. map() — Given the array below, create a new array with 10% tax added to each price.

```
const prices = [100, 250, 75, 300];
```

JavaScript Code

```
// Question 1: map()
// Goal: Create a new array with 10% tax added to each price.
// map() builds a brand-new array by transforming every item in the original array.

const prices = [100, 250, 75, 300];

// Multiply every price by 1.1 to add 10% tax, without changing the original array.
const pricesWithTax = prices.map((price) => price * 1.1);

console.log(pricesWithTax);
```

Expected Output

```
[ 110.00000000000001, 275, 82.5, 330 ]
```

Output Screenshot



Question 2

2. filter() — Filter out words longer than 4 letters.

```
const words = ['cat', 'elephant', 'dog', 'hippopotamus', 'ant'];
```

JavaScript Code

```
// Question 2: filter()
// Goal: Filter out words longer than 4 letters (keep only words of length 4 or less).
// filter() builds a new array containing only the items that pass a test.

const words = ['cat', 'elephant', 'dog', 'hippopotamus', 'ant'];

// Keep only the words whose length is 4 or less.
const shortWords = words.filter((word) => word.length <= 4);

console.log(shortWords);
```

Expected Output

```
[ 'cat', 'dog', 'ant' ]
```

Output Screenshot



Question 3

3. reduce() — Find the total bill amount (qty × price, summed).

```
const cart = [
  { item: 'pen', qty: 2, price: 10 },
  { item: 'book', qty: 1, price: 150 }
];
```

JavaScript Code

```
// Question 3: reduce()
// Goal: Find the total bill amount (quantity x price, summed across all items).
// reduce() combines all array items into a single value (here, a running total).

const cart = [
  { item: 'pen', qty: 2, price: 10 },
  { item: 'book', qty: 1, price: 150 },
];

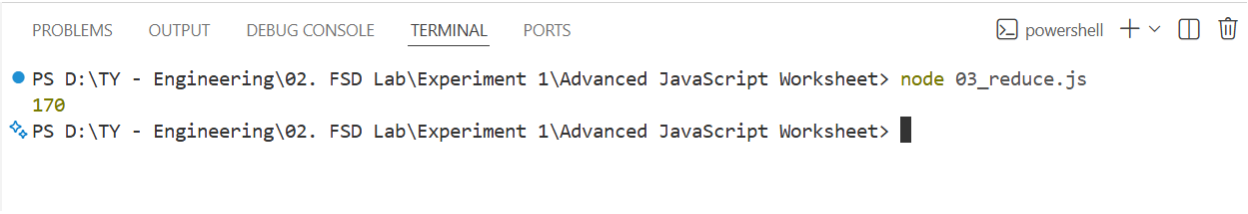
// Start the running total at 0, then add (qty * price) for every item in the cart.
const totalBill = cart.reduce((runningTotal, cartItem) => {
  return runningTotal + cartItem.qty * cartItem.price;
}, 0);
```

```
console.log(totalBill);
```

Expected Output

```
170
```

Output Screenshot



Question 4

4. forEach() — Print each student's name in uppercase.

```
const students = ['ravi', 'sita', 'aman'];
```

JavaScript Code

```
// Question 4: forEach()
// Goal: Print each student's name in uppercase.
// forEach() runs a function once for every item in the array (it does not return a new array).

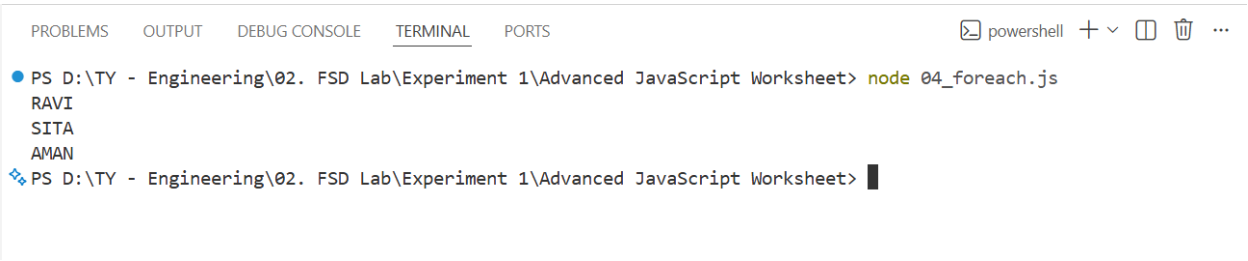
const students = ['ravi', 'sita', 'aman'];

// Loop through each student and print their name in uppercase letters.
students.forEach((studentName) => {
  console.log(studentName.toUpperCase());
});
```

Expected Output

```
RAVI
SITA
AMAN
```

Output Screenshot



Question 5

5. some() / every() — Check (a) if any student failed (< 40), (b) if all students passed (>= 40).

```
const marks = [45, 67, 89, 32, 55];
```

JavaScript Code

```
// Question 5: some() and every()
// Goal:
//   (a) Check if ANY student failed (marks < 40) using some().
//   (b) Check if ALL students passed (marks >= 40) using every().

const marks = [45, 67, 89, 32, 55];

// some() returns true if at least one item passes the test.
const didAnyStudentFail = marks.some((mark) => mark < 40);

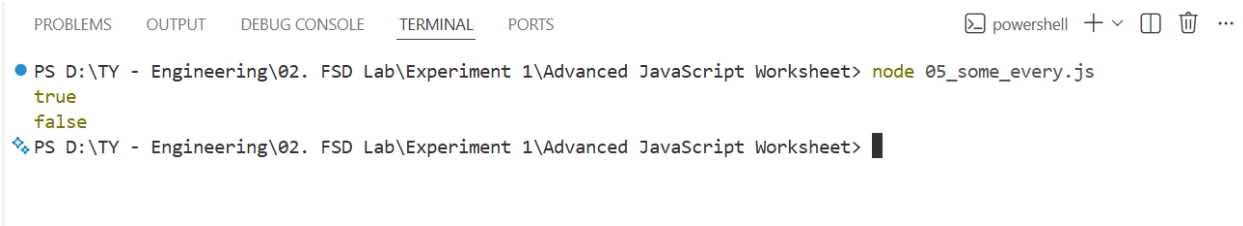
// every() returns true only if ALL items pass the test.
const didAllStudentsPass = marks.every((mark) => mark >= 40);

console.log(didAnyStudentFail);
console.log(didAllStudentsPass);
```

Expected Output

```
true
false
```

Output Screenshot



Question 6

6. find() / findIndex() — Find the user with id === 2, and their index in the array.

```
const users = [{id:1,name:'A'},{id:2,name:'B'},{id:3,name:'C'}];
```

JavaScript Code

```
// Question 6: find() and findIndex()
// Goal: Find the user with id === 2, and their index in the array.

const users = [
  { id: 1, name: 'A' },
```

```
  { id: 2, name: 'B' },
  { id: 3, name: 'C' },
];

// find() returns the first matching item itself.
const userWithId2 = users.find((user) => user.id === 2);

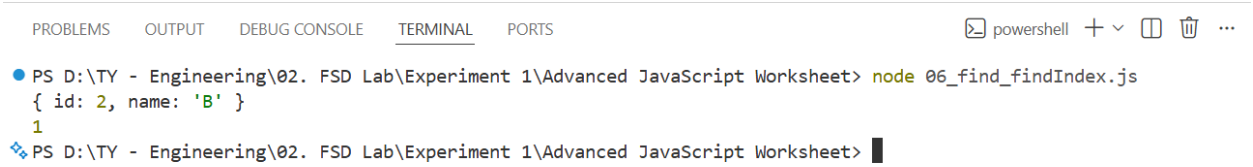
// findIndex() returns the position (index) of the first matching item.
const indexOfUserWithId2 = users.findIndex((user) => user.id === 2);

console.log(userWithId2);
console.log(indexOfUserWithId2);
```

Expected Output

```
{ id: 2, name: 'B' }
1
```

Output Screenshot



Question 7

7. slice() vs splice() — (a) Get a copy of just Tue-Thu using slice. (b) Remove 'Wed' using splice. Confirm slice does not modify the original array.

```
const days = ['Mon','Tue','Wed','Thu','Fri'];
```

JavaScript Code

```
// Question 7: slice() vs splice()
// Goal:
//   (a) Get a copy of just Tue-Thu using slice() (does NOT change the original array).
//   (b) Remove 'Wed' using splice() (DOES change the original array).
//   (c) Confirm that slice() did not modify the original array.

const days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri'];

// slice(1, 4) copies items from index 1 up to (but not including) index 4.
const midWeekDays = days.slice(1, 4);
console.log(midWeekDays); // ['Tue', 'Wed', 'Thu']

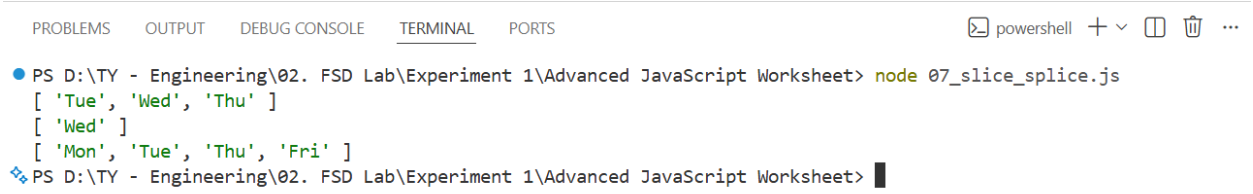
// splice(2, 1) removes 1 item starting at index 2, and changes the original array.
const removedDays = days.splice(2, 1);
console.log(removedDays); // ['Wed'] - the item(s) that were removed
```

```
// The original array is now permanently changed, proving splice() mutates the array.
console.log(days); // ['Mon', 'Tue', 'Thu', 'Fri']
```

Expected Output

```
[ 'Tue', 'Wed', 'Thu' ]
[ 'Wed' ]
[ 'Mon', 'Tue', 'Thu', 'Fri' ]
```

Output Screenshot



Object Methods

Question 8

8. Object.keys() / values() / entries() — Print the keys, values, and entries of the object below, separately.

```
const car = { brand: 'Toyota', model: 'Corolla', year: 2022 };
```

JavaScript Code

```
// Question 8: Object.keys() / Object.values() / Object.entries()
// Goal: Print the keys, values, and entries of the car object, separately.

const car = { brand: 'Toyota', model: 'Corolla', year: 2022 };

// Object.keys() returns an array of just the property names.
console.log(Object.keys(car));

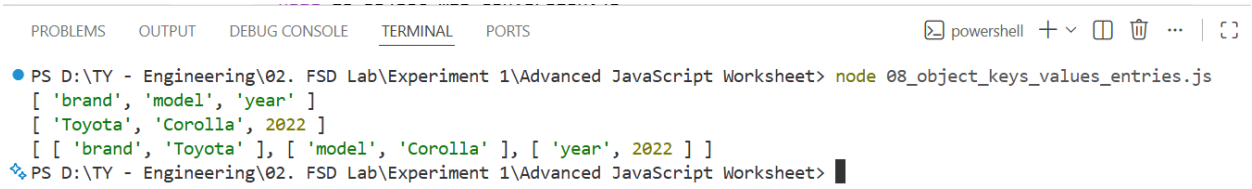
// Object.values() returns an array of just the property values.
console.log(Object.values(car));

// Object.entries() returns an array of [key, value] pairs.
console.log(Object.entries(car));
```

Expected Output

```
[ 'brand', 'model', 'year' ]
[ 'Toyota', 'Corolla', 2022 ]
[ [ 'brand', 'Toyota' ], [ 'model', 'Corolla' ], [ 'year', 2022 ] ]
```

Output Screenshot



Question 9

9. Object.assign() — Merge defaults with userPrefs into a new settings object.

```
const defaults = { theme: 'light', fontSize: 14 };
const userPrefs = { fontSize: 18 };
```

JavaScript Code

```
// Question 9: Object.assign()
// Goal: Merge defaults with userPrefs into a new settings object.

const defaults = { theme: 'light', fontSize: 14 };
const userPrefs = { fontSize: 18 };

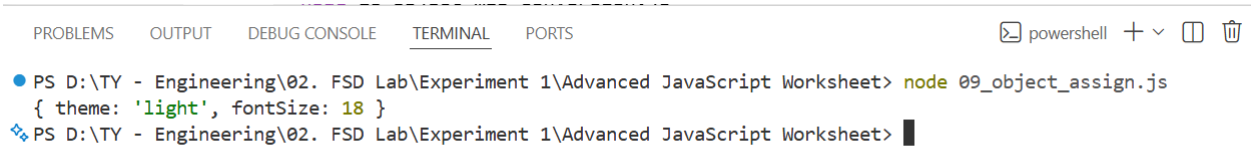
// Object.assign(target, ...sources) copies properties from source objects onto the target.
// Using {} as the target keeps "defaults" and "userPrefs" unchanged.
// Later sources overwrite matching keys from earlier ones, so userPrefs.fontSize wins.
const settings = Object.assign({}, defaults, userPrefs);

console.log(settings);
```

Expected Output

```
{ theme: 'light', fontSize: 18 }
```

Output Screenshot



Question 10

10. Spread operator — Combine a and b into one object using ... . Predict the output before running it.

```
const a = { x: 1, y: 2 };
const b = { y: 5, z: 9 };
```

JavaScript Code

```
// Question 10: Spread operator (...)
// Goal: Combine object "a" and object "b" into one object using the spread operator.
// Predict the output before running it: since "b" is spread AFTER "a",
// any matching keys in "b" will overwrite the ones from "a".

const a = { x: 1, y: 2 };
const b = { y: 5, z: 9 };

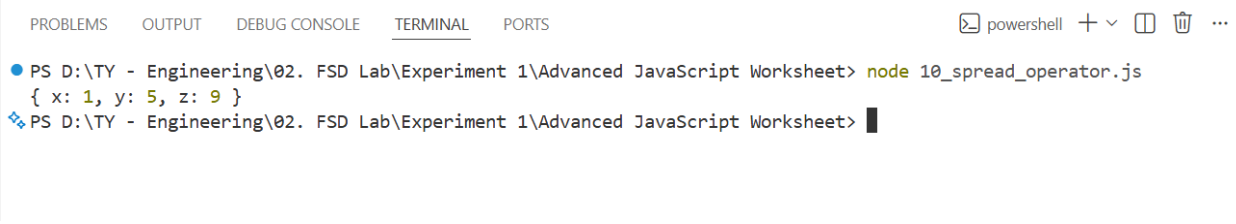
// Spreading copies each object's own properties into a brand-new object.
const merged = { ...a, ...b };

console.log(merged); // b.y (5) overwrites a.y (2) because b is spread later
```

Expected Output

```
{ x: 1, y: 5, z: 9 }
```

Output Screenshot



Combining Arrays and Objects

Question 11

11. Given the employees array, use map to get an array of names, filter to get employees in the 'Sales' dept, and reduce to get the total salary of the whole company.

```
const employees = [
  { name: 'Ravi', dept: 'Sales', salary: 30000 },
  { name: 'Sita', dept: 'IT', salary: 45000 },
  { name: 'Aman', dept: 'Sales', salary: 32000 }
];
```

JavaScript Code

```
// Question 11: Combining Arrays and Objects (map + filter + reduce)
// Goal:
//   (a) Use map() to get an array of employee names.
//   (b) Use filter() to get employees in the 'Sales' department.
//   (c) Use reduce() to get the total salary of the whole company.

const employees = [
  { name: 'Ravi', dept: 'Sales', salary: 30000 },
  { name: 'Sita', dept: 'IT', salary: 45000 },
  { name: 'Aman', dept: 'Sales', salary: 32000 },
];

// (a) Pull out just the "name" field from every employee.
const employeeNames = employees.map((employee) => employee.name);

// (b) Keep only the employees who work in the Sales department.
const salesTeam = employees.filter((employee) => employee.dept === 'Sales');

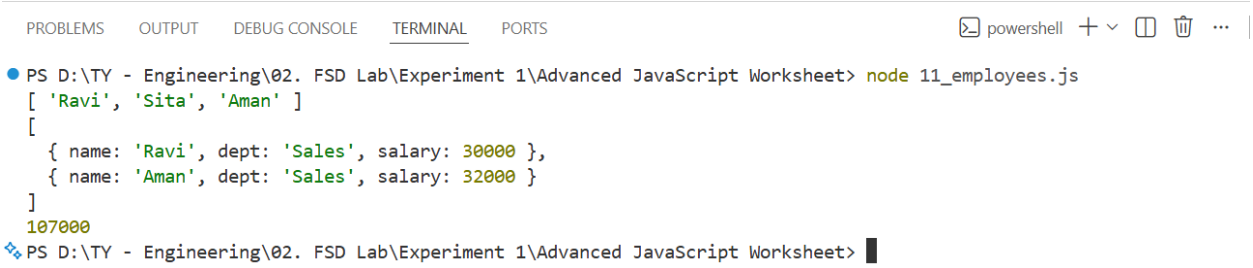
// (c) Add up every employee's salary to get the company's total salary cost.
const totalSalary = employees.reduce((sum, employee) => sum + employee.salary, 0);

console.log(employeeNames);
console.log(salesTeam);
console.log(totalSalary);
```

Expected Output

```
[ 'Ravi', 'Sita', 'Aman' ]
[
  { name: 'Ravi', dept: 'Sales', salary: 30000 },
  { name: 'Aman', dept: 'Sales', salary: 32000 }
]
107000
```

Output Screenshot



Question 12

12. Given inventory, use find to get the first out-of-stock item, and Object.keys to list the fields of one item.

```
const inventory = [
  { item: 'Pen', stock: 0 },
  { item: 'Book', stock: 5 }
];
```

JavaScript Code

```
// Question 12: Combining Arrays and Objects (find + Object.keys)
// Goal:
//   (a) Use find() to get the first out-of-stock item.
//   (b) Use Object.keys() to list the fields of one item.

const inventory = [
  { item: 'Pen', stock: 0 },
  { item: 'Book', stock: 5 },
];

// find() returns the first item whose stock is exactly 0 (out of stock).
const firstOutOfStockItem = inventory.find((product) => product.stock === 0);

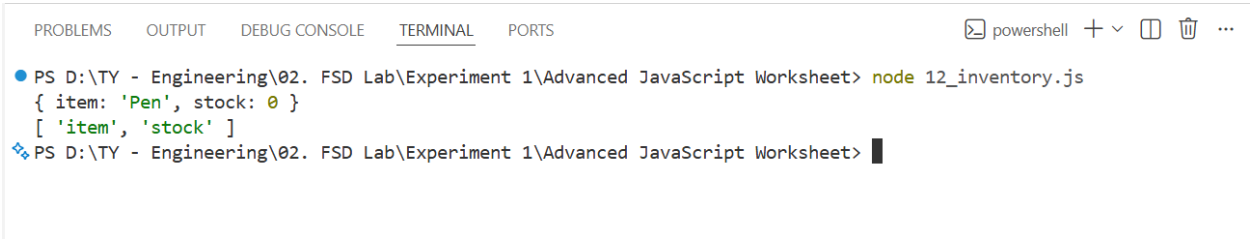
// Object.keys() lists the property names of the first item in the inventory.
const inventoryItemFields = Object.keys(inventory[0]);

console.log(firstOutOfStockItem);
console.log(inventoryItemFields);
```

Expected Output

```
{ item: 'Pen', stock: 0 }
[ 'item', 'stock' ]
```

Output Screenshot



Map

Question 13

13. Create a Map storing roll numbers (keys) and names (values). Add 3 entries, check if roll number 102 exists, then delete it.

JavaScript Code

```
// Question 13: Map basics
// Goal: Create a Map storing roll numbers (keys) and names (values).
// Add 3 entries, check if roll number 102 exists, then delete it.

// A Map is like an object, but keys can be any type and insertion order is preserved.
const rollNumberToName = new Map();

// Add three roll-number -> name entries.
rollNumberToName.set(101, 'Ravi');
rollNumberToName.set(102, 'Sita');
rollNumberToName.set(103, 'Aman');

// has() checks whether a key exists in the Map.
console.log(rollNumberToName.has(102)); // true

// delete() removes the entry with the given key.
rollNumberToName.delete(102);

// Confirm that roll number 102 no longer exists after deletion.
console.log(rollNumberToName.has(102)); // false
```

Expected Output

```
true
false
```

Output Screenshot



Question 14

14. Use chaining (.set().set().set()) to build a Map of fruit -> price in a single statement.

JavaScript Code

```
// Question 14: Map chaining
// Goal: Use chaining (.set().set().set()) to build a Map of fruit -> price in one statement.
// Each call to .set() returns the Map itself, so calls can be chained together.

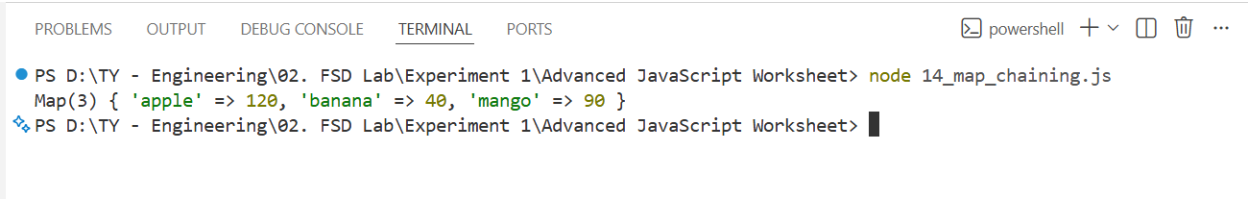
const fruitPrices = new Map()
  .set('apple', 120)
  .set('banana', 40)
  .set('mango', 90);

console.log(fruitPrices);
```

Expected Output

```
Map(3) { 'apple' => 120, 'banana' => 40, 'mango' => 90 }
```

Output Screenshot



Question 15

15. Loop over your Map using for...of and print "key: value" for each entry. Then repeat using .forEach().

JavaScript Code

```
// Question 15: Iterating over a Map
// Goal: Loop over a Map using for...of and print "key: value" for each entry.
// Then repeat the same thing using .forEach().

const fruitPrices = new Map()
  .set('apple', 120)
  .set('banana', 40)
  .set('mango', 90);

// Method 1: for...of gives you a [key, value] pair on each loop iteration.
for (const [fruitName, price] of fruitPrices) {
  console.log(`${fruitName}: ${price}`);
}

// Method 2: forEach() gives you (value, key) as separate parameters, in that order.
```

```
fruitPrices.forEach((price, fruitName) => {
  console.log(`${fruitName}: ${price}`);
});
```

Expected Output

```
apple: 120
banana: 40
mango: 90
apple: 120
banana: 40
mango: 90
```

Output Screenshot



Question 16

16. Convert the object below into a Map using Object.entries(), then convert it back to a plain object using Object.fromEntries().

```
const obj = { a: 1, b: 2, c: 3 };
```

JavaScript Code

```
// Question 16: Object <-> Map conversion
// Goal: Convert the object below into a Map using Object.entries(),
// then convert it back to a plain object using Object.fromEntries().

const obj = { a: 1, b: 2, c: 3 };

// Object.entries(obj) turns the object into an array of [key, value] pairs,
// which the Map constructor can then use directly.
const objAsMap = new Map(Object.entries(obj));
console.log(objAsMap);

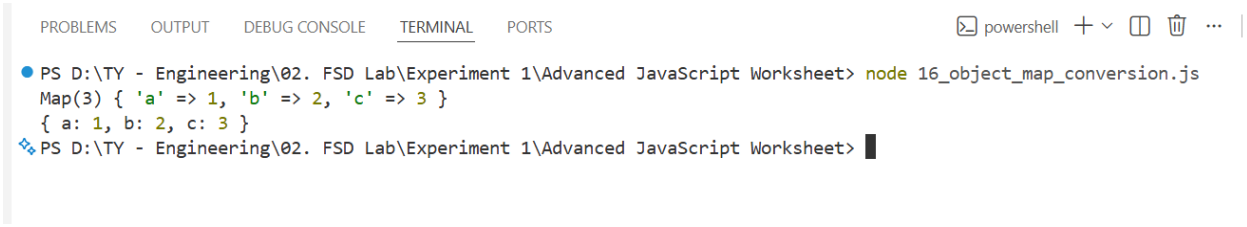
// Object.fromEntries() does the reverse: it turns [key, value] pairs back into an object.
const backToPlainObject = Object.fromEntries(objAsMap);
console.log(backToPlainObject);
```

Expected Output

```
Map(3) { 'a' => 1, 'b' => 2, 'c' => 3 }
```

```
{ a: 1, b: 2, c: 3 }
```

Output Screenshot



Set

Question 17

17. Given the visitors array, use a Set to get only unique names, then convert the Set back to an array.

```
const visitors = ['Ravi','Sita','Ravi','Aman','Sita'];
```

JavaScript Code

```
// Question 17: Unique values with Set
// Goal: Use a Set to get only unique visitor names, then convert the Set back to an array.

const visitors = ['Ravi', 'Sita', 'Ravi', 'Aman', 'Sita'];

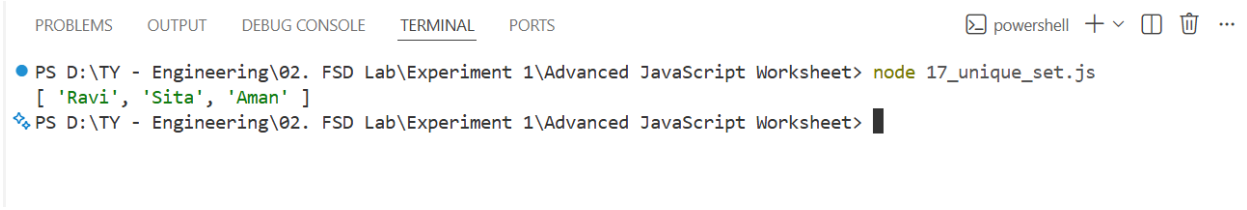
// A Set automatically drops duplicate values, keeping only unique ones.
// Spreading the Set (...) turns it back into a regular array.
const uniqueVisitors = [...new Set(visitors)];

console.log(uniqueVisitors);
```

Expected Output

```
[ 'Ravi', 'Sita', 'Aman' ]
```

Output Screenshot



Question 18

18. Write a function hasDuplicates(arr) that uses Set.size vs arr.length to check if an array has duplicates.

JavaScript Code

```
// Question 18: hasDuplicates()
// Goal: Write a function hasDuplicates(arr) that uses Set.size vs arr.length
// to check whether an array contains any duplicate values.

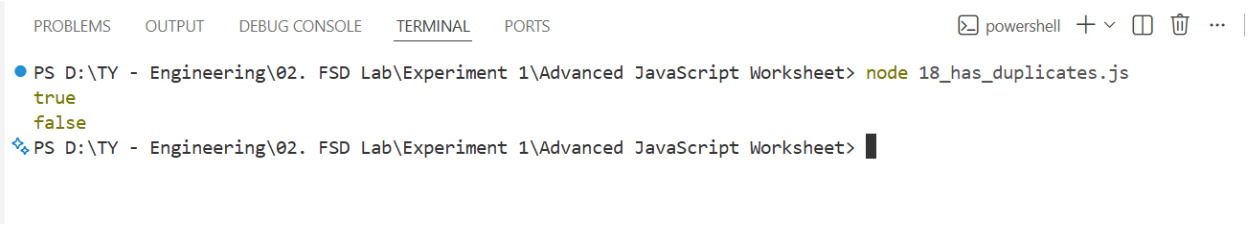
function hasDuplicates(arr) {
  // A Set can only hold unique values, so if duplicates were removed,
  // the Set's size will be smaller than the original array's length.
  return new Set(arr).size !== arr.length;
}

console.log(hasDuplicates([1, 2, 3, 2])); // true - the 2 appears twice
console.log(hasDuplicates([1, 2, 3])); // false - all values are unique
```

Expected Output

```
true
false
```

Output Screenshot



Question 19

19. Given setA and setB below, find their union and intersection.

```
const setA = [1, 2, 3, 4];
const setB = [3, 4, 5, 6];
```

JavaScript Code

```
// Question 19: Union and Intersection of two Sets
// Goal: Given setA and setB, find their union (all unique values from both)
// and their intersection (only values that appear in both).

const setA = [1, 2, 3, 4];
const setB = [3, 4, 5, 6];

// Union: combine both arrays, then use a Set to remove duplicates.
const union = [...new Set([...setA, ...setB])];
```

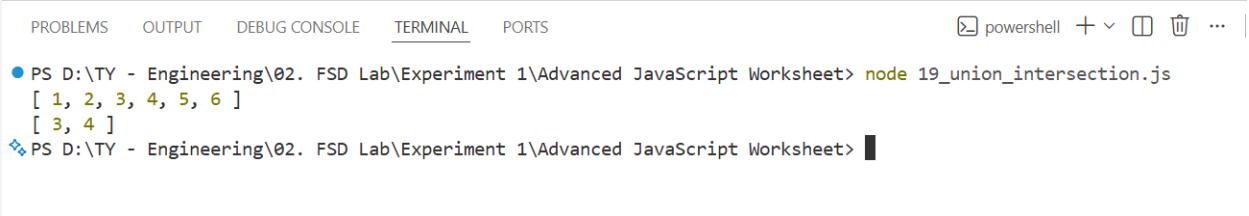
```
console.log(union);

// Intersection: keep only the values from setA that also exist in setB.
const intersection = setA.filter((number) => setB.includes(number));
console.log(intersection);
```

Expected Output

```
[ 1, 2, 3, 4, 5, 6 ]
[ 3, 4 ]
```

Output Screenshot



Mini Challenge (Advanced)

Question 20

20. Using the orders array below:

a) Find the total amount of 'delivered' orders (filter + reduce).

b) Get a unique list of customer names (map + Set).

c) Build a Map of customer -> total amount spent, counting only delivered orders.

```
const orders = [
  { id: 1, customer: 'Ravi', amount: 500, status: 'delivered' },
  { id: 2, customer: 'Sita', amount: 1200, status: 'pending' },
  { id: 3, customer: 'Ravi', amount: 300, status: 'delivered' },
  { id: 4, customer: 'Aman', amount: 800, status: 'cancelled' }
];
```

JavaScript Code

```
// Question 20: Mini Challenge - orders
// Goal, using the orders array below:
//   (a) Find the total amount of 'delivered' orders (filter + reduce).
//   (b) Get a unique list of customer names (map + Set).
//   (c) Build a Map of customer -> total amount spent, counting only delivered orders.

const orders = [
  { id: 1, customer: 'Ravi', amount: 500, status: 'delivered' },
  { id: 2, customer: 'Sita', amount: 1200, status: 'pending' },
```

```
  { id: 3, customer: 'Ravi', amount: 300, status: 'delivered' },
  { id: 4, customer: 'Aman', amount: 800, status: 'cancelled' },
];

// (a) Keep only delivered orders, then add up their amounts.
const totalDeliveredAmount = orders
  .filter((order) => order.status === 'delivered')
  .reduce((sum, order) => sum + order.amount, 0);
console.log(totalDeliveredAmount);

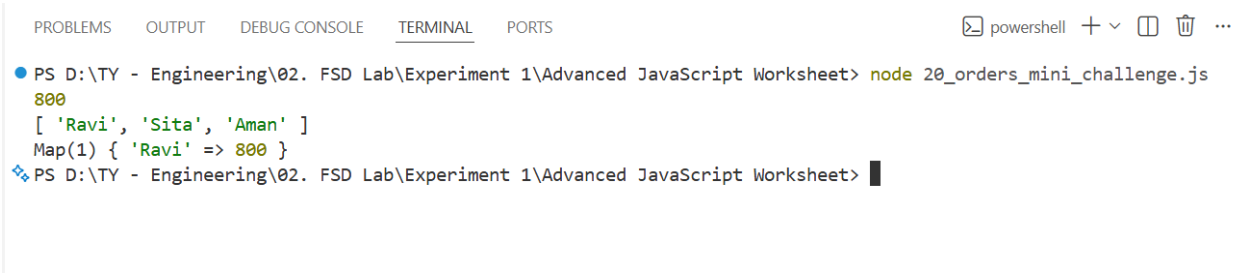
// (b) Pull out every customer name, then use a Set to remove duplicates.
const uniqueCustomers = [...new Set(orders.map((order) => order.customer))];
console.log(uniqueCustomers);

// (c) Build a Map that tracks how much each customer spent on delivered orders only.
const customerTotals = new Map();
orders
  .filter((order) => order.status === 'delivered')
  .forEach((order) => {
    const previousTotal = customerTotals.get(order.customer) || 0;
    customerTotals.set(order.customer, previousTotal + order.amount);
  });
console.log(customerTotals);
```

Expected Output

```
800
[ 'Ravi', 'Sita', 'Aman' ]
Map(1) { 'Ravi' => 800 }
```

Output Screenshot



Steps for execution:

- Open Visual Studio Code.
- Open the Experiment 1 project folder.
- Open the integrated terminal (`Ctrl + ``).
- Navigate to the experiment folder using the cd command.
- Execute the required JavaScript program using:
  node filename.js
- Verify the output displayed in the terminal.
- Capture the output screenshot and paste it into the document.

Conclusion:

This experiment helped me understand the fundamentals of JavaScript programming, including variables, operators, conditionals, loops, arrays, and strings. I successfully implemented and executed various programs in Node.js using VS Code, which improved my programming logic and familiarity with JavaScript basics.